

1. Software Test Guide

HYBRID_APP_ADA

Document Control	
Author(s)	Michael Gardner
Version	2.1.0
Status	Released
Status Date	2025-12-14
SPDX-License-Identifier	BSD-3-Clause
License File	See the LICENSE file in the project root
Copyright	© 2025 Michael Gardner, A Bit of Help, Inc.

Project Profile	
Variant	application
Library Role	N/A
Application Role	cli
Assurance	spark-targeted
Execution	sequential
Parallelism	none
Platform	desktop, server
Execution Environment	linux, macos, windows
Processor Architecture	amd64, arm64
Deployment	native

Table of Contents

1. Software Test Guide	1
2. Introduction	1
2.1. Purpose	1
2.2. Scope	1
2.3. References	1
3. Test Strategy	1
3.1. Testing Levels	1
3.2. Testing Philosophy	1
4. Test Organization	1
4.1. Directory Structure	1
4.2. Naming Conventions	2
4.3. Architectural Coverage Intent	2
5. Custom Test Framework	2
5.1. Framework Overview	2
5.2. Framework Usage Pattern	2
5.3. Rationale	3
6. Test Execution	3
6.1. Primary Commands	3
6.2. Direct Execution	3
6.3. Expected Behavior	3
7. Test Details	3
7.1. Unit Tests	3
7.1.1. Domain Tests	3
7.1.2. Application Tests	3
7.1.3. Combinator Tests	3
7.2. Integration Tests	3
7.3. End-to-End Tests	3
7.4. Architecture Validation Tests	3
8. Coverage and Quality Gates	4
8.1. Coverage Expectations	4
8.2. Success Criteria	4
8.3. Platform Considerations	4
9. Writing New Tests	4
9.1. Adding a New Unit or Integration Test	4
9.2. Adding a New End-to-End Test	4
9.3. Regression Tests	4
10. Traceability	4
10.1. Requirements to Test Categories	4
10.2. Detailed Traceability	4
11. Test Maintenance	5
11.1. When to Update the Suite	5
11.2. Maintenance Guidelines	5
11.3. Continuous Integration	5
12. Appendices	6
12.1. Change History	6

2. Introduction

2.1. Purpose

This Software Test Guide (STG) describes the testing approach, test organization, execution procedures, and maintenance guidance for **HYBRID_APP_ADA**.

2.2. Scope

This document covers:

- unit, integration, and end-to-end testing
- custom test framework design and usage
- test execution through make targets and direct commands
- architecture validation tests
- coverage analysis guidance
- maintenance procedures for the test suite

2.3. References

- Software Requirements Specification (SRS)
- Software Design Specification (SDS)
- docs/guides/architecture_enforcement.md

3. Test Strategy

3.1. Testing Levels

Level	Location	Purpose
Unit Tests	test/unit/	Test individual components in isolation, especially Domain and Application logic.
Integration Tests	test/integration/	Test cross-layer interactions with real adapters and wiring.
End-to-End Tests	test/e2e/	Test the full application through the CLI as a black-box interface.

See CHANGELOG for current test counts per release.

3.2. Testing Philosophy

- **Test-Driven:** tests are written alongside or before implementation where practical.
- **Railway-Oriented:** both success and error paths are exercised.
- **Comprehensive:** normal, edge, and error cases are all represented.
- **Automated:** all tests are runnable through `make test-all`.
- **Self-Contained:** no external test framework dependency is required.
- **Fast Feedback:** the suite is intended to support frequent local execution during development.

4. Test Organization

4.1. Directory Structure

```
test/
├─ common/
│   └─ test_framework.ads/adb      # Custom test framework
├─ unit/
│   └─ test_domain_error_result.adb
│   └─ test_domain_person.adb
```

```

├── test_application_command_greet.adb
├── test_application_usecase_greet.adb
├── unit_runner.adb
├── unit_tests.gpr
├── integration/
│   ├── test_greet_full_flow.adb
│   ├── test_infrastructure_console_writer.adb
│   ├── integration_runner.adb
│   └── integration_tests.gpr
├── e2e/
│   ├── test_greeter_cli.adb
│   ├── e2e_runner.adb
│   └── e2e_tests.gpr

```

4.2. Naming Conventions

Element	Convention	Example
Framework file	test_framework.ads/adb	test/common/test_framework.ads
Project file	<category>_tests.gpr	e2e_tests.gpr
Runner	<category>_runner.adb	integration_runner.adb
Test file	test_<layer>_<component>.adb	test_application_usecase_greet.adb

4.3. Architectural Coverage Intent

The test organization reflects the same hybrid architecture family used by the application itself:

- Domain and Application are heavily exercised through unit tests.
- Infrastructure wiring is validated through integration tests.
- Presentation and Bootstrap are validated through end-to-end execution.
- scripts/arch_guard.py validates direct dependency rules during developer workflow and CI/CD.

5. Custom Test Framework

5.1. Framework Overview

The custom lightweight framework in test/common/test_framework.ads provides:

- result registration and aggregation
- per-category summary output
- grand total reporting across categories
- deterministic pass/fail reporting without external dependencies

5.2. Framework Usage Pattern

```

procedure Test_My_Component is
  Total_Tests : Natural := 0;
  Passed_Tests : Natural := 0;

  procedure Run_Test (Name : String; Passed : Boolean) is
  begin
    Total_Tests := Total_Tests + 1;
    if Passed then
      Passed_Tests := Passed_Tests + 1;
    end if;
  end Run_Test;

begin
  Run_Test ("Feature works", Condition);

```

```
Test_Framework.Register_Results (Total_Tests, Passed_Tests);  
end Test_My_Component;
```

5.3. Rationale

A custom framework keeps the project self-contained, simple to understand, and aligned with the educational purpose of the starter while still supporting repeatable automated testing.

6. Test Execution

6.1. Primary Commands

```
make test-all  
make test-unit  
make test-integration  
make test-e2e
```

6.2. Direct Execution

Test categories may also be executed through the relevant GPR project and runner executable when direct troubleshooting is needed.

6.3. Expected Behavior

- unit tests validate isolated logic and data handling
- integration tests validate wiring and adapter behavior
- e2e tests validate complete user-visible CLI behavior and exit codes

7. Test Details

7.1. Unit Tests

7.1.1. Domain Tests

- test_domain_error_result.adb — tests Domain.Error.Result
- test_domain_person.adb — tests Domain.Value_Object.Person

7.1.2. Application Tests

- test_application_command_greet.adb — tests command DTO behavior
- test_application_usecase_greet.adb — tests the greet use case with controlled dependencies

7.1.3. Combinator Tests

The application test suite should include coverage for Result-monad combinators and related functional control-flow helpers where those abstractions are part of the design.

7.2. Integration Tests

- test_greet_full_flow.adb — tests the complete greeting flow through real wiring
- test_infrastructure_console_writer.adb — tests console-writer adapter behavior

7.3. End-to-End Tests

- test_greeter_cli.adb — tests the application through its CLI boundary
- verifies end-to-end user scenarios and exit codes
- stdout detail validation is primarily handled in lower-level tests where appropriate

7.4. Architecture Validation Tests

scripts/arch_guard.py validates direct dependency rules for the application architecture profile. This validation is run during normal developer workflow and again in CI/CD.

8. Coverage and Quality Gates

8.1. Coverage Expectations

Coverage analysis may be performed with GNATcoverage or equivalent tooling appropriate to the host environment. Exact target values are maintained outside this guide when they change frequently.

8.2. Success Criteria

A healthy test run should demonstrate:

- passing unit tests
- passing integration tests
- passing e2e tests
- passing architecture validation
- no unresolved regressions in supported paths

8.3. Platform Considerations

Platform-specific test behavior, if any, shall be documented explicitly in project CHANGELOG or supporting guides rather than embedded as stale metrics here.

9. Writing New Tests

9.1. Adding a New Unit or Integration Test

- create the test source using the naming convention
- add the executable to the relevant GPR file
- register results through the shared framework
- place the test in the category that best reflects the behavior under test

9.2. Adding a New End-to-End Test

- validate a user-visible scenario through the CLI boundary
- focus on observable behavior and exit status
- avoid duplicating fine-grained validation already covered in lower-level tests

9.3. Regression Tests

When defects are fixed, add or update tests so the corrected behavior is permanently covered.

10. Traceability

10.1. Requirements to Test Categories

Requirement Area	Primary Test Coverage
Domain validation and Result handling	Unit tests
Application use cases and DTO behavior	Unit tests and integration tests
Infrastructure adapter behavior	Integration tests
Presentation / CLI behavior	End-to-end tests
Architecture boundary enforcement	<code>scripts/arch_guard.py</code>

10.2. Detailed Traceability

A detailed requirement-to-test mapping may be expanded project-locally as requirements stabilize or as additional verification evidence is needed.

11. Test Maintenance

11.1. When to Update the Suite

- when new packages or features are added
- when public CLI behavior changes
- when bug fixes require regression protection
- when architecture rules or boundaries change
- when test tooling or execution targets change

11.2. Maintenance Guidelines

- keep test names descriptive
- keep tests independent where practical
- reset mock or shared state between test cases
- prefer deterministic behavior over timing-sensitive assertions
- avoid pushing volatile count metrics into this guide

11.3. Continuous Integration

The full validation path should run in CI/CD, including:

- build
- unit tests
- integration tests
- e2e tests
- architecture validation

12. Appendices

12.1. Change History

Version	Date	Author	Changes
2.1.0	2025-12-14	Michael Gardner	Remove hardcoded metrics per documentation standards; metrics now in CHANGELOG.
2.0.0	2025-12-09	Michael Gardner	Complete regeneration for v2.0.0; added combinator and platform coverage guidance.
1.0.0	2025-11-29	Michael Gardner	Initial release.